

Guide de présentation du code

CyberCopilot — Assistant SOC intelligent basé sur LLM

Document de soutenance — Présentation technique
Projet de validation — Année B2

Préparation (2 minutes avant)

Ouvrir un terminal et préparer un environnement propre :

```
cd /home/sir30/soc-assistant  
rm -f data/incidents.db data/llm_cache.json  
clear
```

Ouvrir en parallèle :

- Le terminal (pour la démonstration en direct)
 - L'éditeur de code (VSCode) sur le dossier `soc-assistant`
 - La page GitHub : <https://github.com/abdoul-latif-dev/cybercopilot>
-

1. Introduction (1 minute)

« Je vais vous présenter CyberCopilot, un assistant SOC basé sur les modèles de langage. Le principe est simple : une entreprise nous fournit ses journaux de logs, notre programme détecte automatiquement les attaques, filtre les faux positifs, et propose des actions concrètes à l'analyste. C'est un copilote : il assiste l'humain, mais c'est l'humain qui prend les décisions finales. »

2. Démontrer la qualité — les tests (1 minute)

À taper :

```
pytest tests/ -v
```

À dire :

« Le projet est couvert par 92 tests automatiques qui s'exécutent en moins d'une seconde. Ils valident le parsing des logs, la détection des 6 types d'attaques, l'anonymisation, le cache et le stockage. C'est une garantie de qualité et de non-régression : si une modification casse quelque chose, on le sait immédiatement. »

Résultat attendu : 92 passed in 0.07s

3. La démonstration en direct (5 minutes)

À taper :

```
python app.py
```

Séquence à suivre dans le menu interactif :

Étape	Touche	Discours
1	14	« J'active d'abord l'anonymisation pour la sécurité : les IP internes seront masquées avant tout envoi au LLM. »
2	1	« Je lance la détection d'intrusions SSH. Le programme lit les logs, repère une attaque par force brute depuis une IP russe connue malveillante, et propose des actions. »
3	1	« En tant qu'analyste, je marque l'incident comme traité. Cette décision est tracée en base pour l'audit. »
4	2	« Détection d'injection SQL : il reconnaît les patterns comme ' OR 1=1 . »
5	4	« Détection DDoS : un volume massif de requêtes depuis une même IP. »
6	5	« Et le programme analyse aussi les logs Windows au format JSON, le standard des SIEMs modernes. »
7	7	« Voici le tableau récapitulatif, avec le statut de chaque incident. »
8	10	« Les statistiques montrent l'économie de tokens réalisée par la compression. »
9	0	« Je quitte proprement. »

Phrase clé à placer pendant la démonstration :

« Vous voyez le tag "mode dégradé" : cela signifie que le programme fonctionne même sans clé API. Si le service LLM tombe, il bascule automatiquement sur des règles statiques. C'est essentiel pour un outil de cybersécurité, qui ne doit jamais s'arrêter. »

4. Le tour du code (5 minutes)

Ouvrir VSCode et présenter la structure.

L'architecture modulaire

« Le code est organisé en 6 modules indépendants, articulés en pipeline. Chacun a une responsabilité unique, ce qui permet de remplacer un composant sans toucher aux autres. »

```
src/
├─ parser.py          → lit et extrait les logs (4 formats)
├─ detector.py        → détecte les 6 types d'attaques
├─ enricher.py        → ajoute géolocalisation et réputation des IP
├─ llm_client.py      → appelle le LLM + mode dégradé
├─ prompts.py         → templates de prompts
├─ compressor.py      → économise les tokens (compression)
├─ cache.py           → évite les appels redondants
├─ anonymizer.py      → anonymisation RGPD
├─ storage.py         → base SQLite
└─ cli.py             → affichage terminal
```

Trois fichiers à montrer

src/detector.py — montrer une fonction de détection :

« Voici comment on détecte le brute force : on compte les échecs de connexion par adresse IP, et au-delà de 5 tentatives, on lève une alerte. Pour ajouter un 7e type d'attaque, il suffit d'ajouter une fonction ici. L'architecture est extensible. »

src/llm_client.py — montrer la fonction `fallback_analysis()` :

« Voici le mode dégradé. Si l'appel au LLM échoue, cette fonction génère un résumé à partir de templates prédéfinis. L'outil reste opérationnel quoi qu'il arrive. »

`src/anonymizer.py` — montrer la fonction `anonymize_ip()` :

« La sécurité est au cœur du projet. Cette fonction hash les IP internes avant tout envoi au LLM, pour ne jamais exposer l'infrastructure du client à un fournisseur externe. »

5. Présenter GitHub (2 minutes)

Ouvrir <https://github.com/abdoul-latif-dev/cybercopilot>

« Tout le code est versionné sur GitHub, avec un README complet, les 92 tests, et un dossier `docs/` qui contient les 6 livrables au format PDF. L'historique Git montre l'évolution du projet commit par commit. »

Montrer :

- Le **README** (badges, fonctionnalités, schéma d'architecture)
- Le dossier `docs/` (les livrables consultables)
- L'onglet **Commits** (l'historique de développement)

6. Ce que nous avons DÉPASSÉ du cahier des charges

C'est le point le plus important à mettre en avant. Le projet ne s'est pas contenté de respecter le cahier des charges : il l'a dépassé sur presque tous les axes.

Tableau comparatif

Élément	Promis dans le cahier des charges	Réalisé dans le prototype	Dépassement
Types d'attaques détectées	5	6	+1 (DDoS)
Formats de logs supportés	3	4	+1 (Windows Event Log)
Tests unitaires	Non chiffré (pytest mentionné)	92 tests	Bonus complet
Anonymisation RGPD	Mentionnée	Implémentée et testée	Fonctionnel
Mode dégradé	Mentionné	Implémenté et automatique	Fonctionnel
Workflow analyste	Non prévu	Menu d'actions + traçabilité	Nouveauté
Interface	Terminal simple	Menu interactif + CLI	Double interface
Optimisation tokens	Mentionnée	67 à 94 % mesurés	Mesuré et prouvé
Suppression RGPD	Mentionnée	Commandes delete et purge	Fonctionnel

Détail des dépassements

1. Un 6e type d'attaque : le DDoS

Le cahier des charges prévoyait 5 attaques (brute force, injection SQL, scan de ports, accès admin, horaire anormal). Nous avons ajouté la détection des attaques par déni de service distribué (DDoS), en repérant un volume massif de requêtes HTTP depuis une même IP.

2. Un 4e format de logs : Windows Event Log

Le cahier des charges prévoyait 3 formats (Syslog SSH, Apache, firewall). Nous avons ajouté le support des logs Windows au format JSON, qui est le standard des SIEMs modernes. Le programme reconnaît les Event IDs critiques de Windows (4625 pour les échecs de connexion, 4720 pour la création de comptes, etc.).

3. Un workflow analyste complet

Au-delà de la simple détection, nous avons ajouté un menu d'actions qui permet à l'analyste de marquer chaque incident comme traité, faux positif, ou passé. Chaque décision est tracée en base de données avec un horodatage, ce qui répond à l'exigence de journalisation et d'auditabilité (section 8.6 du cahier des charges).

4. Une double interface

Le cahier des charges prévoyait une interface terminal. Nous avons développé deux interfaces complémentaires : une interface en ligne de commande classique (`main.py`) pour l'automatisation, et un menu interactif (`app.py`) pour l'utilisation manuelle et la démonstration.

5. Une optimisation des coûts mesurée

Le cahier des charges mentionnait l'optimisation. Nous avons implémenté trois mécanismes (compression, cache, modèle économique) et mesuré l'économie réelle : de 67 % sur les attaques par force brute jusqu'à 94 % sur les logs très répétitifs.

6. 92 tests unitaires

Le cahier des charges mentionnait pytest sans chiffre. Nous avons écrit 92 tests qui couvrent tous les modules critiques et passent en moins d'une seconde.

Phrase de synthèse

« Nous n'avons pas seulement respecté le cahier des charges, nous l'avons dépassé. On avait promis 5 attaques, on en détecte 6. On avait promis 3 formats de logs, on en supporte 4. On a ajouté 92 tests, l'anonymisation RGPD complète, le mode dégradé, et un workflow analyste avec traçabilité. Le tout est sur GitHub, testé et documenté. »

7. Ce qui reste volontairement hors périmètre

Il est important de montrer qu'on maîtrise aussi nos limites, et que ce sont des choix assumés.

Hors périmètre	Pourquoi c'est un choix délibéré
Connexion directe à un SIEM en production	Nécessite une infrastructure lourde ; mentionné dans la roadmap
Blocage automatique d'IP	Risque qu'une IA hallucinante bloque un client légitime
Forensics avancé	Domaine d'outils spécialisés (niveau L3)
Interface web	Prévu en évolution ; le terminal suffit pour le prototype
Fine-tuning / RAG	Le prompt engineering classique suffit et reste portable

« Ces exclusions ne sont pas des oublis, ce sont des choix de conception. On a préféré livrer un MVP complet et fiable plutôt qu'un projet trop ambitieux et mal fini. »

8. Questions probables du jury et réponses

Question	Réponse
« Ça tourne sous Windows ? »	« Oui, c'est du Python, ça tourne sur Windows, Linux et macOS. Et ça analyse même les logs Windows au format JSON. »
« Pourquoi pas de blocage automatique ? »	« Choix délibéré. Une IA peut halluciner et bloquer un vrai client. On reste un copilote : l'humain valide toujours. C'est l'approche de Microsoft Security Copilot. »
« Comment vous gérez les hallucinations ? »	« Trois mécanismes : format JSON strict imposé au LLM, validation humaine obligatoire, et croisement avec des règles statiques. »
« Pourquoi SQLite et pas PostgreSQL ? »	« Pour le prototype, SQLite suffit : pas de serveur, un seul fichier. En production, on migrerait vers PostgreSQL ; l'architecture le permet sans refonte. »
« Comment vous économisez les tokens ? »	« Trois techniques : compression des logs répétitifs jusqu'à 94 %, cache des analyses, et choix d'un modèle économique. »
« C'est conforme au RGPD ? »	« Oui : anonymisation activable, droit à l'effacement, permissions chmod 600 sur la base, et minimisation des données envoyées. »
« Combien de fichiers / lignes de code ? »	« 21 fichiers Python, 11 modules, 8 fichiers de tests pour 92 tests. »
« C'est joli mais ça marche vraiment ? »	« Oui, et je vais vous le démontrer en direct. » (lancer la démo)

9. Chiffres clés à connaître par cœur

Métrique	Valeur
Tests pytest	92
Types d'attaques	6
Formats de logs	4
Modules Python	11
Fichiers Python	21
Économie de tokens	67 à 94 %
Permissions base	chmod 600
Temps d'exécution des tests	< 0,1 seconde

10. Cheat sheet (à garder sous les yeux)

PRÉPARATION

```
cd /home/sir30/soc-assistant
rm -f data/incidents.db data/llm_cache.json
clear
```

DÉMONSTRATION

```
pytest tests/ -v          → 92 passed
python app.py
14 → anonymisation ON
1  → brute force SSH (puis 1 = marquer traité)
2  → injection SQL
4  → DDoS
5  → Windows Event Log
7  → liste des incidents
10 → statistiques tokens
0  → quitter
```

DÉPASSEMENTS DU CAHIER DES CHARGES

6 attaques (vs 5) · 4 formats (vs 3)
92 tests · anonymisation · mode dégradé · workflow analyste

MESSAGE CLÉ

"Copilote, pas remplaçant. L'humain décide toujours."

Conseils de présentation

- Parler lentement, surtout pendant la démonstration.
- Laisser le jury lire l'écran 2 à 3 secondes avant de cliquer.
- Mettre en avant les dépassements du cahier des charges (DDoS, Windows, anonymisation).
- En cas de question difficile : « C'est un excellent point, c'est dans notre roadmap. »
- Rester humble mais confiant : le projet est solide et testé.